

FD03-EPIS-Informe Especificación Requerimientos



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Informe de Especificación de Requerimientos

Sistema Analizador de Dependencias Multi-Lenguaje (DepAnalyzer)

Curso: Calidad y Pruebas de Software

Docente: Patrick Cuadros Quiroga

Integrantes:

Carbajal Vargas, Andre Alejandro (2023077287)

Yupa Gómez, Fátima Sofía (2023076618)

Tacna - Perú

2026

Sistema *Analizador de Dependencias Multi-Lenguaje (DepAnalyzer)*

Informe de Especificación de Requerimientos

Versión *1.1*

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-04-22	Versión inicial
1.1	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-06-23	Unificación del formato institucional

ÍNDICE GENERAL

1. Introducción
2. Generalidades de la Empresa
 1. Nombre de la Empresa
 2. Visión
 3. Misión
 4. Organigrama
3. Visionamiento de la Empresa
 1. Descripción del problema
 2. Objetivo de Negocios
 3. Objetivo de diseño
 4. Alcance del proyecto
 5. Viabilidad del sistema
 6. Información obtenida del Levantamiento de información
4. Análisis de procesos
 1. Diagrama de Procesos Actual
 2. Diagrama de Procesos Propuesto
5. Especificación de Requerimientos de Software
 1. Cuadro de Requerimientos funcionales Inicial
 2. Cuadro de Requerimientos no funcionales
 3. Cuadro de Requerimientos funcionales Final
 4. Regla de Negocio
6. Fase de Desarrollo
 1. Perfil del Usuario
 2. Modelo Conceptual
 1. Diagrama de paquetes
 2. Diagrama de casos de uso
 3. Escenarios de casos de uso (narrativas)
 3. Modelo Lógico
 1. Análisis de Objetos
 2. Diagrama de Actividades con objetos
 3. Diagrama de secuencia

4. Diagrama de clases
7. Conclusiones
8. Recomendaciones
9. Bibliografía
10. Webgrafía

1. Introducción

El presente Informe de Especificación de Requerimientos de Software (ERS) define, de manera verificable, las capacidades funcionales y no funcionales del sistema *DepAnalyzer*. Este documento integra la visión académica del proyecto (FD02), la factibilidad evaluada (FD01) y el estado actual del código fuente para asegurar trazabilidad real entre requisitos, implementación y pruebas.

El sistema está orientado a analizar proyectos Maven, Gradle, npm y Python, identificar dependencias desactualizadas, detectar vulnerabilidades CVE y brindar mecanismos de actualización guiada en terminal, con confirmación explícita y respaldo de archivos de build.

2. Generalidades de la Empresa

2.1 Nombre de la empresa

Universidad Privada de Tacna – Facultad de Ingeniería – Escuela Profesional de Ingeniería de Sistemas.

2.2 Visión

Formar profesionales líderes, innovadores y comprometidos con la calidad, capaces de desarrollar soluciones tecnológicas aplicadas a problemas reales del entorno.

2.3 Misión

Brindar formación integral en ingeniería de software, promoviendo investigación aplicada, ética profesional y producción de software de calidad con impacto académico y social.

2.4 Organigrama

flowchart TD

```
UPT[Universidad Privada de Tacna] --> FI[Facultad de Ingeniería]
FI --> EPIS[Escuela Profesional de Ingeniería de Sistemas]
EPIS --> CURSO[Curso: Calidad y Pruebas de Software]
CURSO --> DOC[Docente Evaluador]
CURSO --> EQ[Equipo del Proyecto DepAnalyzer]
```

3. Visionamiento de la Empresa

3.1 Descripción del problema

En proyectos modernos, la gestión manual de dependencias presenta dos fallas frecuentes: (a) atraso de versiones con deuda técnica acumulada y (b) exposición a CVEs en dependencias directas y transitivas. La revisión manual no escala para árboles de dependencias medianos o grandes y dificulta tomar decisiones oportunas de mantenimiento.

3.2 Objetivo de negocios

Reducir tiempo de diagnóstico técnico y riesgo de seguridad en proyectos Maven, Gradle, npm y Python, proporcionando una herramienta CLI/TUI que entregue evidencia rápida, trazable y accionable para mantenimiento de dependencias.

3.3 Objetivo de diseño

Diseñar una solución modular en Kotlin/JVM que:

- detecte automáticamente tipo de proyecto,
- parsee dependencias y repositorios,
- consulte versiones y CVEs (OSS Index y enriquecimiento NVD opcional),
- emita reportes legibles y JSON,
- y habilite actualización interactiva con respaldo previo.

3.4 Alcance del proyecto

Incluido en la versión actual:

- Análisis de proyectos Maven (`pom.xml`), Gradle Groovy (`build.gradle`) y Gradle Kotlin (`build.gradle.kts`).
- Comandos CLI `analyze`, `tui` y `update`.
- Detección de desactualización por consulta de metadatos de repositorios.
- Detección de CVEs con OSS Index, NVD o modo automático (`--oss`, `--nvd`).
- Clasificación de vulnerabilidades directas/transitivas y renderizado en árbol.
- Exportación de resultados en JSON (`--output json`).
- Actualización guiada con confirmación interactiva y backup automático (`.bak`).

Fuera de alcance en la versión actual:

- Interfaz gráfica web/escritorio.
- Actualización automática sin confirmación del usuario.
- Reemplazo total de suites SCA empresariales de pago.

3.5 Viabilidad del sistema

Con base en FD01, la viabilidad del sistema es **alta** en dimensiones técnica, operativa, legal, social y ambiental. El costo operativo del entorno académico es bajo, y la pila tecnológica empleada es open-source, madura y documentada.

3.6 Informacion obtenida del levantamiento de informacion

Fuentes de entrada para este ERS:

- Documento de factibilidad (docs/FD01-Informe-Factibilidad.md).
- Documento de visión (docs/FD02-Informe-Vision.md).
- Código fuente del sistema (módulos cli, core, parser, repository, report, update, tui).
- README y workflows de CI/CD del repositorio.

4. Analisis de procesos

4.1 Diagrama de procesos actual

Proceso manual sin herramienta integrada.

flowchart LR

```

    A[Desarrollador revisa build files] --> B[Busca versiones manualmente en repositorios]
    B --> C[Consulta CVEs en sitios externos]
    C --> D[Consolida hallazgos manualmente]
    D --> E[Decide actualizaciones sin trazabilidad uniforme]
  
```

4.2 Diagrama de procesos propuesto

Proceso automatizado con DepAnalyzer.

flowchart LR

```

    A[Usuario ejecuta analyze o tui] --> B[ProjectDetector identifica tipo de proyecto]
    B --> C[Parsers extraen dependencias y repositorios]
    C --> D[ProjectAnalyzer consulta últimas versiones]
    D --> E[OssIndexClient detecta CVEs]
    E --> F{--nvd?}
    F -- Sí --> G[NvdClient enriquece y fusiona CVEs]
    F -- No --> H[Continuar con OSS Index]
    G --> I[ReportGenerator y ConsoleRenderer]
    H --> I
  
```

I --> J[Salida consola/JSON]
 J --> K[Usuario ejecuta update para remediación guiada]
 K --> L[Backup build file y aplicación confirmada]

5. Especificacion de Requerimientos de Software

5.1 Cuadro de requerimientos funcionales inicial

ID	Requerimiento funcional inicial	Criterio general de aceptación
RFI-01	Detectar tipo de proyecto Java	Identifica Maven, Gradle Groovy o Gradle Kotlin con archivos estándar
RFI-02	Extraer dependencias declaradas	Obtiene groupId:artifactId:version según parser correspondiente
RFI-03	Extraer repositorios del proyecto	Lee repos configurados para consultar versiones
RFI-04	Detectar desactualizaciones	Compara versión actual vs versión más reciente
RFI-05	Detectar CVEs	Consulta vulnerabilidades por dependencia
RFI-06	Clasificar vulnerabilidades	Separa vulnerabilidades directas y transitivas
RFI-07	Mostrar resultados por consola	Presenta resumen legible y árbol de problemas
RFI-08	Exportar JSON	Genera reporte estructurado para automatización

5.2 Cuadro de requerimientos no funcionales

ID	Requerimiento no funcional	Métrica / Umbral	Evidencia esperada
RNF-01	Portabilidad	Ejecución en Windows, Linux y macOS	Scripts generados por installDist y release nativo
RNF-02	Usabilidad técnica	Comandos con ayuda y flags consistentes	--help, documentación en README
RNF-03	Confiabilidad	Manejo controlado de	No abortar todo el flujo por fallo parcial de fuente externa

ID	Requerimiento no funcional	Métrica / Umbral	Evidencia esperada
RNF-04	Seguridad de credenciales	errores de red/parseo Solo enviar credenciales a hosts confiables por allowlist	DEPANALYZER_TRUSTED_CREDENTIAL_HOSTS aplicado
RNF-05	Interoperabilidad	JSON válido y estable para consumo externo	--output json parseable por scripts
RNF-06	Rendimiento	Tiempo razonable en proyectos medianos (objetivo <= 30s local)	Corridas de referencia y pruebas internas
RNF-07	Mantenibilidad	Arquitectura modular y tipada	Separación de paquetes y cobertura de tests
RNF-08	Auditabilidad	Evidencia repetible en CI/CD	Workflows de test y release definidos

5.3 Cuadro de requerimientos funcionales final

ID	Requerimiento funcional final	Prioridad	Trazabilidad técnica (módulo/código)
RF-01	Detectar automáticamente tipo de proyecto (pom.xml, build.gradle, build.gradle.kts)	Alta	parser/ProjectDetector.kt
RF-02	Parsear dependencias Maven y Gradle	Alta	parser/PomDependencyParser.kt, parser/GradleGroovyDependencyParser.kt, parser/GradleKotlinDependencyParser.kt
RF-03	Parsear repositorios del proyecto	Alta	parser/GradleRepositoryParser.kt, parser/PomDependencyParser.kt
RF-04	Consultar versión más reciente por repositorio	Alta	repository/RepositoryClient.kt, core/ProjectAnalyzer.kt
RF-05	Detectar CVEs usando OSS Index	Alta	repository/OssIndexClient.kt, core/ProjectAnalyzer.kt
RF-06	Consultar CVEs con NVD	Media	repository/NvdClient.kt, repository/VulnerabilityMerger.kt, flag -

ID	Requerimiento funcional final	Prioridad	Trazabilidad técnica (módulo/código)
			-nvd
RF-07	Clasificar vulnerabilidades directas y transitivas	Alta	core/ProjectAnalyzer.kt, report/DependencyReport.kt
RF-08	Mostrar resultados en consola con árbol de dependencias	Alta	report/ConsoleRenderer.kt, report/DependencyTreeBuilder.kt
RF-09	Exportar resultados a JSON	Alta	report/ReportGenerator.kt, opción --output json
RF-10	Ofrecer interfaz TUI interactiva	Media	tui/AnalyzeTuiApp.kt, comando tui y flag --tui
RF-11	Ejecutar actualización guiada con selección interactiva	Alta	cli/UpdateCommand.kt, update/UpdatePlanner.kt
RF-12	Respaldar build file antes de aplicar cambios	Alta	update/BuildFileBackup.kt
RF-13	Simular cambios sin modificar archivos (--dry-run)	Media	cli/UpdateCommand.kt
RF-14	Fallar proceso en CI ante CVEs críticos (--fail-on-critical)	Media	cli/DepAnalyzerCli.kt

5.4 Regla de negocio

ID	Regla de negocio	Aplicación
RN-01	Ninguna actualización de dependencia se aplica sin confirmación interactiva del usuario	Flujo update y flujo de actualización en TUI
RN-02	Antes de modificar pom.xml / build.gradle / build.gradle.kts se genera backup .bak	BuildFileBackup.ensureBackup
RN-03	En caso de error de fuente externa (OSS Index/NVD), el análisis continúa con degradación controlada	Manejo de excepciones en ProjectAnalyzer

ID	Regla de negocio	Aplicación
RN-04	Si se usan credenciales de repositorio, solo se envían a destinos HTTPS confiables explícitos	InputSafety.isTrustedCredentialDestination
RN-05	En conflictos de token OSS Index, el token CLI tiene prioridad sobre variable de entorno	--oss-token vs OSS_INDEX_TOKEN
RN-06	El modo TUI prioriza cobertura dinámica de transitivas para análisis interactivo	Forzado de análisis dinámico en TUI

6. Fase de Desarrollo

6.1 Perfil del usuario

Perfil	Características	Necesidades principales
Usuario técnico básico	Usa terminal para comandos directos	Diagnóstico rápido en consola
Usuario técnico intermedio	Integra herramientas con scripts	Salida JSON y parámetros de control
Usuario técnico avanzado/CI	Automatiza control de calidad y seguridad	Exit code controlado y reportes estables
Mantenedor académico	Evalúa trazabilidad y evidencia	Coherencia entre documentos, código y pruebas

6.2 Modelo Conceptual

6.2.1 Diagrama de paquetes

```
flowchart TD
    CLI[com.depanalyzer.cli] --> CORE[com.depanalyzer.core]
    CLI --> TUI[com.depanalyzer.tui]
    CLI --> UPDATE[com.depanalyzer.update]
    CORE --> PARSER[com.depanalyzer.parser]
    CORE --> REPO[com.depanalyzer.repository]
    CORE --> REPORT[com.depanalyzer.report]
    CORE --> GRAPH[com.depanalyzer.core.graph]
    UPDATE --> CORE
    UPDATE --> PARSER
    REPO --> SECURITY[com.depanalyzer.security]
```

6.2.2 Diagrama de casos de uso

```
flowchart LR
    U[Usuario] --> UC1[Analizar proyecto]
    U --> UC2[Exportar reporte JSON]
    U --> UC3[Visualizar TUI]
    U --> UC4[Actualizar dependencias guiadas]
    U --> UC5[Ejecutar dry-run]
    U --> UC6[Fallar build por CVE crítico]
    UC1 --> S[DepAnalyzer]
    UC2 --> S
    UC3 --> S
    UC4 --> S
    UC5 --> S
    UC6 --> S
```

6.2.3 Escenarios de casos de uso (narrativas)

Caso de uso	Actor	Flujo principal	Resultado
CU-01 Analizar proyecto	Usuario	Ejecuta analyze <ruta>; el sistema detecta tipo, parsea, consulta versiones/CVEs y renderiza	Reporte en consola con estado de dependencias
CU-02 Exportar JSON	Usuario	Ejecuta analyze <ruta> --output json	Se genera dependency-report.json
CU-03 Analizar con TUI	Usuario	Ejecuta tui <ruta> o analyze --tui; el sistema muestra progreso y paneles interactivos	Visualización interactiva de hallazgos
CU-04 Actualizar dependencias	Usuario	Ejecuta update <ruta>; selecciona sugerencias; confirma	Build file actualizado y backup creado
CU-05 Simular actualización	Usuario	Ejecuta update --dry-run; selecciona sugerencias	Resumen de cambios simulados sin modificar archivos

6.3 Modelo Lógico

6.3.1 Analisis de objetos

Objeto	Responsabilidad	Tipo
ProjectAnalyzer	Orquestrar análisis de dependencias, versiones y vulnerabilidades	Control
ProjectDetector	Detectar tipo de proyecto por archivos build	Entidad de soporte

Objeto	Responsabilidad	Tipo
PomDependencyParser / Gradle*DependencyParser	Extraer dependencias y metadatos por formato	Servicio parser
RepositoryClient	Consultar metadatos de versión en repositorios	Servicio infraestructura
OssIndexClient / NvdClient	Consultar vulnerabilidades y enriquecer CVEs	Servicio integración externa
DependencyReport	Representar resultado consolidado de análisis	DTO de dominio
ReportGenerator / ConsoleRenderer	Transformar reporte a JSON o salida legible	Presentación
AnalyzerUpdatePlanner	Planificar sugerencias de actualización	Lógica de negocio
BuildFileUpdater (+ implementaciones)	Aplicar cambios por tipo de build file	Estrategia
BuildFileBackup	Crear respaldo previo a modificaciones	Seguridad operativa

6.3.2 Diagrama de actividades con objetos

flowchart TD

```

A([Inicio]) --> B[Usuario ejecuta analyze]
B --> C[ProjectDetector.detect]
C --> D[Parser correspondiente extrae dependencias]
D --> E[RepositoryClient busca versiones recientes]
E --> F[OssIndexClient consulta CVEs]
F --> G{--nvd}
G -- Sí --> H[NvdClient y VulnerabilityMerger]
G -- No --> I[Continuar]
H --> J[ProjectAnalyzer clasifica hallazgos]
I --> J
J --> K[DependencyReport]
K --> L{Salida}
L -- Consola --> M[ConsoleRenderer]
L -- JSON --> N[ReportGenerator]
M --> O([Fin])
N --> O

```

6.3.3 Diagrama de secuencia

sequenceDiagram

```

actor Usuario
participant CLI as DepAnalyzerCli
participant PA as ProjectAnalyzer
participant PD as ProjectDetector
participant Parser as Parser Maven/Gradle
participant RC as RepositoryClient

```

```

participant OI as OssIndexClient
participant NVD as NvdClient
participant RG as ReportGenerator/ConsoleRenderer
Usuario ->> CLI: analyze . --nvd --output json
CLI ->> PA: analyze(request)
PA ->> PD: detect(projectPath)
PD -->> PA: ProjectType
PA ->> Parser: parse(buildFile)
Parser -->> PA: dependencies + repositories
PA ->> RC: getLatestVersion(...)
RC -->> PA: latest versions
PA ->> OI: getVulnerabilities(dependencies)
OI -->> PA: CVEs OSS Index
PA ->> NVD: getVulnerabilities(dependencies)
NVD -->> PA: CVEs NVD
PA -->> CLI: DependencyReport
CLI ->> RG: toJson(report)
RG -->> Usuario: dependency-report.json

```

6.3.4 Diagrama de clases

```

classDiagram
    class Depanalyzer
    class Analyze
    class Tui
    class Update

    class ProjectAnalyzer {
        +analyze(projectDir, ...): DependencyReport
    }

    class ProjectDetector {
        +detect(directory): ProjectType
    }

    class RepositoryClient {
        +getLatestVersion(repository, groupId, artifactId): String?
    }

    class OssIndexClient {
        +getVulnerabilities(dependencies): Map~String, List~ Vulnerability~~
    }

    class NvdClient {
        +getVulnerabilities(dependencies): Map~String, List~ Vulnerability~~
    }

    class DependencyReport {
        +projectName: String
        +upToDate: List~DependencyInfo~
        +outdated: List~OutdatedDependency~
    }

```

```

+directVulnerable: List~VulnerableDependency~
+transitiveVulnerable: List~VulnerableDependency~
+dependencyTree: List~DependencyTreeNode~ ?
}

class AnalyzerUpdatePlanner {
    +plan(projectDir, options): UpdatePlan
}

class BuildFileUpdater {
    <<interface>>
    +applyUpdate(file, suggestion): Boolean
}

class PomBuildFileUpdater
class GradleGroovyBuildFileUpdater
class GradleKotlinBuildFileUpdater

Analyze --> ProjectAnalyzer
Tui --> ProjectAnalyzer
Update --> AnalyzerUpdatePlanner
ProjectAnalyzer --> ProjectDetector
ProjectAnalyzer --> RepositoryClient
ProjectAnalyzer --> OssIndexClient
ProjectAnalyzer --> NvdClient
ProjectAnalyzer --> DependencyReport
AnalyzerUpdatePlanner --> BuildFileUpdater
BuildFileUpdater <|.. PomBuildFileUpdater
BuildFileUpdater <|.. GradleGroovyBuildFileUpdater
BuildFileUpdater <|.. GradleKotlinBuildFileUpdater

```

7. Conclusiones

1. El ERS define requerimientos verificables y trazables al código real del proyecto, reduciendo ambigüedad documental.
2. El núcleo funcional del sistema cumple el objetivo académico: análisis integral de dependencias, versiones y vulnerabilidades.
3. La presencia de update y TUI confirma un alcance de remediación guiada (no automática), con controles de seguridad operativa.
4. Los requisitos no funcionales priorizan portabilidad, seguridad de credenciales, interoperabilidad y auditabilidad en CI/CD.
5. La arquitectura modular facilita mantenimiento, pruebas y evolución incremental del sistema.

8. Recomendaciones

1. Mantener sincronizados FD01, FD02, FD03 y README al cerrar cada sprint o hito académico.
2. Agregar una matriz formal requisito → prueba automatizada para evaluación docente y auditoría interna.
3. Establecer línea base de rendimiento por tipo/tamaño de proyecto para medir regresiones.
4. Fortalecer pruebas de escenarios no estándar de Gradle y repositorios privados con autenticación.
5. Definir en siguiente iteración un historial comparativo entre ejecuciones para seguimiento temporal de riesgo.

9. Bibliografía

1. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*.
2. Sommerville, I. (2016). *Software Engineering*.
3. ISO/IEC 25010:2011. *Systems and software quality models*.
4. Sonatype. (2026). *OSS Index Documentation*.
5. NIST. (2026). *National Vulnerability Database (NVD) Developers Guide*.
6. Apache Software Foundation. (2026). *Maven Documentation*.
7. Gradle Inc. (2026). *Gradle User Manual*.
8. JetBrains. (2026). *Kotlin Documentation*.

10. Webgrafia

- Repositorio del proyecto: README.md
- Configuración build: build.gradle.kts
- CLI principal: src/main/kotlin/com/depanalyzer/cli/DepAnalyzerCli.kt
- Comando de actualización:
src/main/kotlin/com/depanalyzer/cli/UpdateCommand.kt
- Orquestador de análisis:
src/main/kotlin/com/depanalyzer/core/ProjectAnalyzer.kt
- Parsers: src/main/kotlin/com/depanalyzer/parser/
- Repositorios y CVE clients: src/main/kotlin/com/depanalyzer/repository/
- Reportes: src/main/kotlin/com/depanalyzer/report/
- Workflows CI/CD: .github/workflows/test.yml, .github/workflows/release-native.yml
- <https://ossindex.sonatype.org/>
- <https://nvd.nist.gov/developers>
- <https://maven.apache.org/>
- <https://docs.gradle.org/>
- <https://kotlinlang.org/docs/home.html>

11. Historias de Usuario, Criterios y Escenarios BDD

Cada criterio dispone de dos escenarios como mínimo.

HU-01 Detectar el tipo de proyecto

Como desarrollador, **quiero** identificar automáticamente el manifiesto **para** usar el parser correcto.

CA-01.1: reconocer manifiestos soportados.

- **Escenario 01:** Dado un directorio con `pom.xml`, **cuando** se detecta el proyecto, **entonces** el tipo es Maven.
- **Escenario 02:** Dado un directorio con `package.json`, **cuando** se detecta el proyecto, **entonces** el tipo es npm.

CA-01.2: rechazar entradas no soportadas.

- **Escenario 03:** Dado un directorio vacío, **cuando** se detecta el proyecto, **entonces** se informa que no existe un manifiesto reconocido.
- **Escenario 04:** Dado una ruta que es archivo, **cuando** se detecta el proyecto, **entonces** se rechaza por no ser directorio.

HU-02 Analizar vulnerabilidades

Como responsable de mantenimiento, **quiero** conocer CVEs y severidad **para** priorizar correcciones.

CA-02.1: clasificar vulnerabilidades según CVSS.

- **Escenario 05:** Dado un CVSS 9.8, **cuando** se calcula la severidad, **entonces** es CRITICAL.
- **Escenario 06:** Dado un CVSS 7.5, **cuando** se calcula la severidad, **entonces** es HIGH.

CA-02.2: degradar de forma controlada ante fallos externos.

- **Escenario 07:** Dado que OSS falla en modo automático, **cuando** NVD está disponible, **entonces** se utiliza el fallback.
- **Escenario 08:** Dado que se forzó OSS, **cuando** la fuente falla, **entonces** se informa el error sin cambiar de fuente.

HU-03 Exportar evidencia

Como ingeniero de CI, **quiero** consumir JSON estable **para** aplicar políticas automáticas.

CA-03.1: emitir JSON válido y versionado.

- **Escenario 09:** Dado un análisis terminado, **cuando** se solicita JSON, **entonces** contiene `schemaVersion`.
- **Escenario 10:** Dado un reporte sin hallazgos, **cuando** se serializa, **entonces** sus listas siguen siendo parseables.

CA-03.2: comunicar el resultado con códigos de salida.

- **Escenario 11:** Dado un CVE crítico, **cuando** se usa `--fail-on-critical`, **entonces** el proceso retorna 1.
- **Escenario 12:** Dado un error de análisis, **cuando** no puede generarse el reporte, **entonces** retorna 2.

HU-04 Actualizar dependencias con seguridad

Como desarrollador, **quiero** confirmar las actualizaciones **para** evitar cambios destructivos.

CA-04.1: no escribir sin aprobación.

- **Escenario 13:** Dado un plan, **cuando** se ejecuta `--dry-run`, **entonces** el manifiesto no cambia.
- **Escenario 14:** Dado una sugerencia rechazada, **cuando** termina el flujo, **entonces** se conserva la versión.

CA-04.2: proteger el manifiesto.

- **Escenario 15:** Dado una actualización aprobada, **cuando** se escribe, **entonces** existe un archivo `.bak`.
- **Escenario 16:** Dado un resultado inválido, **cuando** se verifica antes de guardar, **entonces** se cancela la escritura.

HU-05 Consultar evidencias públicas

Como docente evaluador, **quiero** navegar documentos y reportes **para** verificar la entrega.

CA-05.1: publicar documentos y manual.

- **Escenario 17:** Dado el portal, **cuando** se abre la portada, **entonces** aparecen FD01 a FD05.
- **Escenario 18:** Dado la portada, **cuando** se selecciona el manual, **entonces** se abre el Manual de Usuario.

CA-05.2: conservar evidencia audiovisual.

- **Escenario 19:** Dado una ejecución Playwright, **cuando** termina un escenario, **entonces** se guarda video WebM.
- **Escenario 20:** Dado una falla, **cuando** Playwright termina, **entonces** conserva captura y traza.

Historia	Escenarios	Evidencia
HU-01	01-04	ProjectDetectorTest y Cucumber
HU-02	05-08	VulnerabilityTest y clientes OSS/NVD
HU-03	09-12	AnalyzeCommandTest y ReportGeneratorTest
HU-04	13-16	UpdateCommandIntegrationTest y updaters

Historia	Escenarios	Evidencia
HU-05	17-20	Playwright, reporte y videos